

An Interpreter for Musical Composition Using Mathlib and Matplotlib

Esteban Alejandro Villalba Delgadillo, Daniel Felipe Barrera Suarez
Systems Engineering
Universidad Distrital Francisco José de Caldas, Bogotá, Colombia

Abstract—Music composition is a domain where rules and structures often guide creativity. This paper presents the design and implementation of a domain-specific language (DSL) that interprets and compiles user-defined musical rules. The solution relies on theoretical foundations from formal languages to build a parser and interpreter, which generate mathematical representations of notes via `mathlib`, and visualizes them using `matplotlib`.

Index Terms—Domain-specific language, formal languages, music composition, interpreter, compiler, `matplotlib`.

I. INTRODUCTION

The intersection of computer science and music composition has enabled the creation of tools that automate or assist in generating musical structures. Traditionally, music software relies on predefined graphical interfaces or MIDI sequencing. However, there is a growing interest in rule-based or algorithmic composition, where the logic behind note generation can be explicitly encoded.

Domain-Specific Languages (DSLs) offer a way to model such systems. They allow users to define grammar-based rules that produce structured musical outputs. This approach benefits from strong theoretical foundations, including formal grammars, automata, and compiler construction.

In this project, we propose the creation of a DSL designed for musical rule interpretation and composition. Drawing from the concepts explored in Workshop 1, including lexical analysis, parsing techniques, and syntactic trees, we implement a system that compiles user-defined rules into musical representations.

Our system uses `mathlib` to convert rules into mathematical note objects, and `matplotlib` to render them graphically, offering a visual feedback mechanism. Related work in this domain includes systems such as Sonic Pi and TidalCycles, which also aim to blend programming and music, but focus on live coding and performance rather than rule-based grammar construction.

Recent tools like Sonic Pi [3], FoxDot [4], and TidalCycles [2] demonstrate the educational and creative potential of code-driven music. While these tools focus on live performance, PianoDSL aims to complement them with a grammar-based system more suitable for analysis, teaching, and rule-based composition.

II. RELATED WORK

Several systems have explored the intersection of code and music. Sonic Pi [3] is a live coding environment aimed

at real-time music creation, mainly used for teaching and live performance. FoxDot [4] builds on similar principles but introduces a more Pythonic syntax. TidalCycles provides a highly expressive system based on Haskell for composing music patterns.

While these systems are designed for live interaction and performance, our approach focuses on rule-based grammar composition. This allows for a more structured parsing process using compiler techniques. Additionally, we aim to provide visual feedback not found in most live coding tools.

Academic efforts such as those described in [1] and [2] show the importance of accessible musical DSLs in education. Our system builds on these ideas but remains accessible to beginners, while also being grounded in formal language theory.

III. METHODS AND MATERIALS

The solution architecture follows a traditional language pipeline: lexical analysis, parsing, and semantic interpretation.

The language was defined using a context-free grammar that supports note definitions, durations, sequences, and repetition. The input consists of a set of rules that define note sequences using symbolic identifiers and operators.

A. Language Design

Tokens such as `NOTE`, `DURATION`, and `REPEAT` are identified in the lexical phase. Parsing produces an abstract syntax tree (AST), which the interpreter uses to evaluate the structure into a mathematical representation.

B. Mathlib Integration

The core of the music generation is based on ‘`mathlib`’, which handles mathematical transformations, including scaling and sequencing of note values. Each parsed rule translates into a function or transformation within ‘`mathlib`’.

C. Graphical Representation

After semantic analysis, the resulting note data is passed to a visualization module implemented with `matplotlib`. This module generates a piano roll diagram, where each note is represented as a horizontal bar.

The horizontal position indicates the start time, and the bar’s length corresponds to the note’s duration. The vertical position reflects the note’s pitch.

The visualization provides immediate feedback to users and helps validate the correctness of rule-based input. Although not interactive, it supports debugging and composition analysis.

D. System Architecture

Our system follows the traditional compiler pipeline, adapted to musical interpretation:

- **Input Stage:** Users write musical commands in DSL format (e.g., `play C4 for 1.0 at 0.0`).
- **Lexical Analysis:** The input is tokenized using regular expressions.
- **Syntactic Parsing:** Tokens are checked against a context-free grammar to form a valid sequence.
- **Semantic Analysis:** Valid instructions are converted into tuples: (`note`, `duration`, `start time`).
- **Mathlib Stage:** Notes are translated into mathematical representations.
- **Visualization:** A piano roll or graphical feedback is generated using Matplotlib.

Figure 1 illustrates this pipeline.

IV. RESULTS

We performed a series of tests to verify the correctness of our interpreter and compiler:

A. Unit Tests

A total of 15 unit tests were written to validate lexical analysis, parser output, and AST interpretation. These include correct token recognition, tree formation for nested rules, and transformation into mathematical expressions.

B. Integration and Acceptance

Integration tests confirmed that rules entered by users are interpreted into expected musical objects and visualized without errors. Acceptance tests with peer users showed that rule syntax is understandable and usable with minimal instruction.

C. Comparison

Although similar to Sonic Pi in purpose, our solution differs by offering a rule-based syntax rather than imperative live-coding commands. A comparison is shown in Table I.

The corresponding piano roll generated from this input is shown in Figure 2.

TABLE I: Comparison with Similar Music DSLs

Feature	This Work	Sonic Pi
Grammar-based rules	Yes	No
Visual rendering	Yes (matplotlib)	No
Math-based representation	Yes (mathlib)	No
Live coding	No	Yes

V. CONCLUSIONS

The integration of `mathlib` enables mathematical manipulation of musical notes. The system already includes basic static visual output using `matplotlib`, which supports note validation and user comprehension.

ACKNOWLEDGMENT

This project was developed as part of the Programming Languages course at Universidad Distrital Francisco José de Caldas. The authors thank Professor Carlos Andrés Sierra for the foundational theoretical guidance provided during Workshop 1.

REFERENCES

- [1] Brown, C., et al. “Designing DSLs for Education.” *Proc. ACM Educators Workshop*, 2019.
- [2] Dannenberg, R. “Music Representation Issues, Techniques, and Systems.” *Computer Music Journal*, 2021.
- [3] Sonic Pi. Available: <https://sonic-pi.net>
- [4] FoxDot. Available: <https://foxdot.org>

PianoDSL - System Architecture

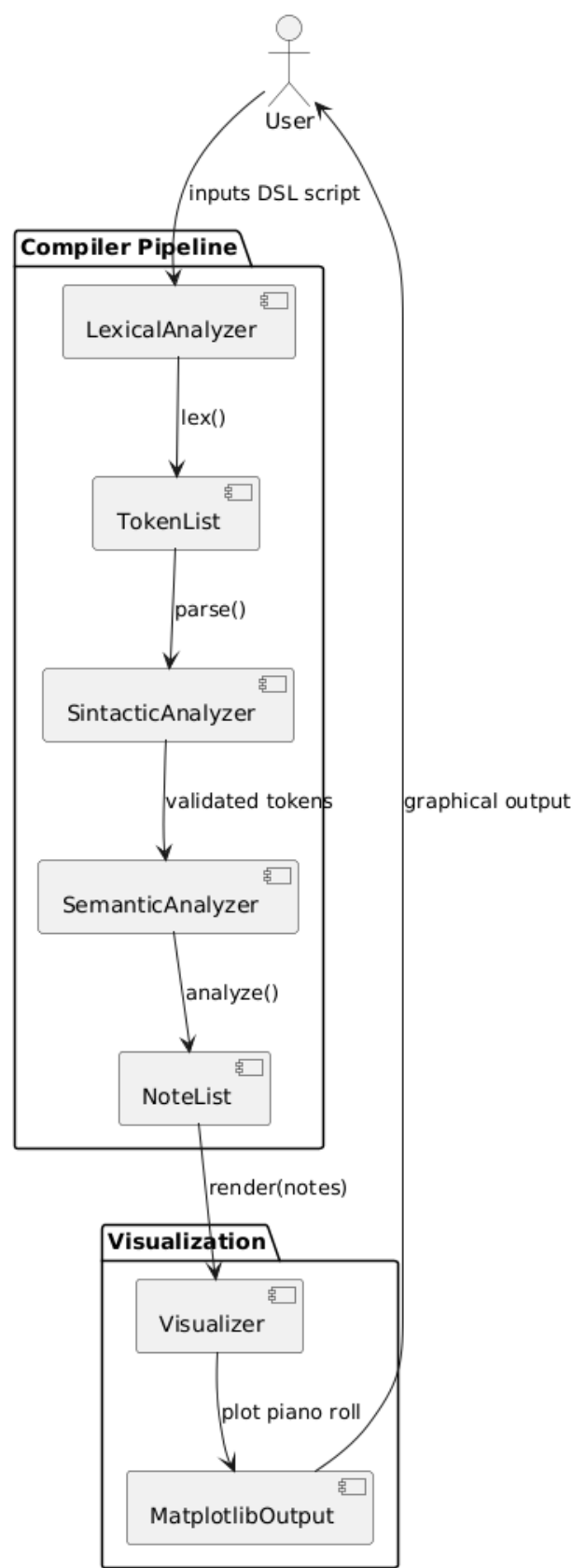


Fig. 1: System architecture of the musical compiler

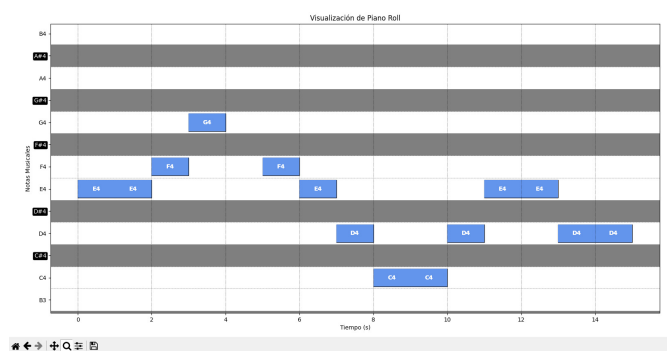


Fig. 2: Piano roll visualization of “Ode to Joy” generated from the DSL interpreter